

Reviewer Pack — Minimal Hodge Theoretical Complexity and Communication Compl...

Entry 57e4310b682b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 17:41:56 UTC

Minimal Hodge Theoretical Complexity and Communication Complexity Rank

Entry ID: 57e4310b682b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 17:41:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every instance of a communication complexity problem with matrix rank r , the minimum number of distinct algebraic cycles in the Hodge decomposition of its associated algebraic variety is $\Omega(r)$.

Rationale (proposer's reasoning):

The Hodge structure provides a geometric encoding of polynomial equations that could capture nontrivial dependencies in complex systems. Communication complexity measures the amount of communication needed to solve certain problems, and if there's a deep connection between these two, it might reveal new insights into computational hardness.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a56ed2f8a228c245

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the minimum number of distinct algebraic cycles in the Hodge decomposition of an associated algebraic variety has a correlation coefficient of at least 0.8 with matrix rank r , across all instances up to rank 40, and no seed instance produces a metric value greater than 10 for the correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "Communication Complexity" AND "Matrix Rank" - "Algebraic Geometry" AND "communication complexity" AND "algebraic cycles" - "Hodge decomposition" AND "minimum number of distinct algebraic cycles" AND "matrix rank"

Top relevant hits considered: - [s2:6f234602ceea6022678a355ae683962b95691547] The intersection of commutative algebra and algebraic geometry with combinatorics and discrete geometry is a rich and ex - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hyper-surfaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0 for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def hodge_decomposition(A):
    n = len(A)
    rank = sum(1 for row in gaussian_elimination(A) if any(row))
    return rank

def generate_communication_complexity_problem(r):
    A = [[random.randint(-10, 10) for _ in range(r)] for _ in range(r)]
    B = [[random.randint(-10, 10) for _ in range(r)] for _ in range(r)]
    C = matrix_multiplication(A, B)
    return C

n_max = 40
instances_tested = 30
metric_values = []

for r in [5, 10, 15, 20, 30, 40]:
    for _ in range(instances_tested):
        A = generate_communication_complexity_problem(r)
        rank = determinant(A)
        if rank == 0:
            continue
        hodge_cycles = hodge_decomposition(A)
        metric_values.append(hodge_cycles / rank)

mean_value = sum(metric_values) / len(metric_values)
std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / len(metric_values))

conjecture_holds = all(0.8 <= value <= 10 for value in metric_values)
counterexample = "" if conjecture_holds else "correlation_coefficient_out_of_bounds"

return {
    "metric_name": "Hodge Cycles to Rank Ratio",
    "metric_value": mean_value,
    "instances_tested": instances_tested * 6,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_out_of_bounds\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support conditions for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13821
2	preregistration	ollama_remote	glm4:latest	0	0	9306
3	novelty	ollama_remote	glm4:latest	0	0	8490
4	novelty	ollama_remote	glm4:latest	0	0	12141
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13446
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9933
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11919
9	judge	ollama_remote	glm4:latest	0	0	13925

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102941 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/57e4310b682b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/57e4310b682b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/57e4310b682b.tar.gz (if generated)